# Blatt 01: Hangman (Wiederholung Swing)

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

*Dieses Blatt ist bewusst einfach gestaltet und dient der Wiederholung Ihrer Kenntnisse aus der LV "Programmieren 1".*

Implementieren Sie in Java das Spiel Hangman.

# Installation JDK und IDE, Deaktivierung AI-Support

Sie benötigen für die Bearbeitung der Übungsaufgaben ein *Java Development Kit* (JDK). Wir verwenden in der Lehrveranstaltung "Programmieren 2" aus verschiedenen Gründen die *Long-Term Support (LTS)*-Variante, d.h. aktuell das "Java SE Development Kit 21 (LTS)" (JDK 21).

Sofern noch nicht geschehen, installieren Sie bitte auf Ihrem Rechner **Java SE 21 (LTS)** in einer *64-bit Version*. Wenn Sie mehrere JDKs installiert haben sollten, stellen Sie bitte sicher, dass Sie für "Programmieren 2" tatsächlich das Java SE 21 (LTS) verwenden. Der Anbieter des JDKs sollte keine Rolle spielen.

Installieren Sie auf Ihrem Rechner eine IDE für Java Ihrer Wahl. Machen Sie sich mit der Arbeitsweise Ihrer IDE vertraut. Wenn Sie im Verlauf des Praktikums feststellen, dass die gewählte IDE nicht für Sie gemacht ist, können Sie jederzeit auf eine andere IDE wechseln.

Da Sie das Programmierhandwerk erlernen und üben und vertiefen sollen, dürfen Sie im Rahmen dieser Lehrveranstaltung noch keine KI-gestützten Assistenten benutzen. Bitte schalten Sie sämtliche KI-Unterstützung wie beispielsweise Copilot, JetBrains AI Assistant, Cursor, CodeGPT, Codeium, Tabnine, Windsurf, ... (Liste nicht vollständig) für die Bearbeitung der Übungsaufgaben in dieser Lehrveranstaltung ab.

# Anlegen eines Java-Projektes

Legen Sie für die Bearbeitung der Aufgabe ein neues Java-Projekt in Ihrer IDE an. Achten Sie bitte darauf, dass im Projektpfad **keine Leerzeichen** und **keine Sonderzeichen** (Umlaute o.ä.) vorkommen! Dies kann zu teilweise seltsamen Fehler führen.

Wir werden in dieser Lehrveranstaltung das Build-Tool **Gradle** verwenden, dieses aber erst später im Verlauf der Lehrveranstaltung besprechen. Für dieses erste Übungsblatt können Sie Ihr Projekt noch so konfigurieren, dass es ohne Build-Tool arbeitet und direkt in der IDE kompiliert und gestartet wird.

Testen Sie bitte die genutzte Java-Version:

1. Konsole: Geben Sie den Befehl `java -version` auf der Konsole ein. Die Ausgabe sollte `java version "21.0.6" 2025-01-21 LTS` (oder ähnlich) ergeben. Wichtig sind die "21" und "LTS".
2. IDE: Erstellen Sie ein Programm, welches die Anweisung `System.out.println(System.getProperty("java.version"));` ausführt. Beim Start über die IDE sollte dabei die Ausgabe `21.0.6` (oder ähnlich) herauskommen. Wichtig ist die "21".

Korrigieren Sie Ihr Setup, wenn Sie andere Ausgaben erhalten.

# Übersetzen und Starten und Debuggen von Programmen

Erstellen Sie in Ihrem neuen Projekt die folgende Klasse:

```java
public class Main {
    public static void main(String... args) {
        String version = System.getProperty("java.version");
        System.out.println(version);
    }
}
```

Worin besteht der Unterschied zwischen dem Übersetzen und dem Starten des obigen Programms? Wie unterscheiden Sie das in Ihrer IDE? Wie können Sie das Programm manuell auf der Konsole übersetzen und starten? Demonstrieren Sie das live im Praktikum.

Worin besteht der Unterschied zwischen Compiler-Warnungen/-Fehlern und Laufzeit-Fehlern? Erklären Sie das im Praktikum an selbst gewählten Beispielen am obigen Programm.

Starten Sie das obige Programm im Debug-Modus Ihrer IDE. Halten Sie die Ausführung am `System.out.println` an, verändern Sie den Wert der Variablen `version` und führen Sie dann die nächste Anweisung (das `System.out.println`) aus. Wie beenden Sie das Programm? Demonstrieren Sie das live im Praktikum.

# Swing und Java2D

Das Spiel soll vollständig über eine in Swing und Java2D realisierte GUI bedient werden:

1. Visualisieren Sie den Zustand des Galgenmännchens per Java2D.
2. Visualisieren Sie den Zustand des zu ratenden Wortes geeignet.
3. Es muss ein Eingabefeld geben, um den nächsten Buchstaben eingeben zu können.
4. Die bisher eingegebenen Buchstaben sollen in der Reihenfolge der Eingabe angezeigt werden.
5. Das Spiel soll per Knopfdruck abgebrochen und neu gestartet werden können.

# Einlesen von Textdateien

Es soll die Möglichkeit geben, eine Textdatei mit zu ratenden Wörtern einzulesen:

1. Auf Anforderung des Nutzers soll ein Auswahldialog erscheinen.
2. Startverzeichnis ist der Ordner, in dem das Spiel gestartet wurde.
3. Es sollen nur Textdateien mit der Endung `.txt` ausgewählt werden können.
4. Es soll nur eine Datei ausgewählt werden können (also keine Mehrfachselektion).
5. Die ausgewählte Textdatei soll eingelesen werden (Format: pro Zeile ein Wort). Alle eingelesenen Wörter sollen in eine Menge überführt und für das Spiel nutzbar gemacht werden.
6. Es soll für jedes neue Spiel ein zufälliges Wort aus der Menge der eingelesenen Wörter zum Raten ausgewählt werden.

# Dokumentation

Erstellen Sie ein UML-Klassendiagramm für Ihre Lösung.

# Fortgeschrittenere Aufgaben

1. Heben Sie in der Anzeige der eingegebenen Buchstaben die korrekt geratenen Buchstaben in grüner Farbe hervor.
2. Passen Sie die Visualisierung des Galgenmännchens farblich an die Tageszeit an, beispielsweise könnte es in den Nachtstunden eine Art Dark-Mode geben und tagsüber einen Light-Mode. Alternativ könnten Sie auch die Farbe der Zeichnung mit der Uhrzeit variieren.
3. Lassen Sie den Nutzer per Slider die Schwierigkeit variieren: Steuerung der Länge des zu ratenden Wortes und/oder Steuerung der Anzahl der erlaubten Fehlversuche.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

***Last modified:*** *c1867fd (tooling: remove numbering of homework tasks, 2025-04-23)*